P2788R0: Linkage for modular constants

Audience: EWG
S. Davis Herring <herring@lanl.gov>
Los Alamos National Laboratory
February 8, 2023

Introduction

National body comment US 8-036 points out an unfortunate interaction between C++98 linkage rules and C++20 modules. This paper explains the interaction in more detail, motivates a choice among the suggestions offered by the comment, and provides wording for that choice.

History

C headers typically use macros to define constants:

```
#define MAX_BUNNIES 57
struct bunny bunnies[MAX_BUNNIES];
```

C++ allows the use of constant integer variables in constant expressions to avoid the macros:

```
const int max_bunnies=57;
bunny bunnies[max_bunnies];
```

If, however, `max_bunnies` appears in a header file, it might end up being defined more than once. In C++17, we can write

```
inline constexpr int max_bunnies=57;
```

to simply allow such multiple definitions. In the absence of the inline variables feature, the first solution adopted was to give such variables internal linkage ([basic.link]/3.2), such that the multiple definitions are of *distinct variables*.

It is little surprise that this implicit duplication of variables causes problems with the one-definition rule. First, it requires a special exception ([basic.def.odr]/14.5.1) to allow

```
const int zero=0;
inline int get_zero() {return zero;}
```

to appear in more than one translation unit despite the fact that the multiple definitions of `get_zero` refer to *different* variables named `zero`. Moreover, odr-using such variables enjoys no such affordance:

```
#include<algorithm>
const int ceiling=5;
inline int cap(int x) {return std::min(x,ceiling);}  // oops
```

`std::min` accepts and returns references, so the different definitions of `cap` odr-use different `ceiling` variables and this is IFNDR. (The trap can be avoided by writing `+ceiling` to produce a temporary, but that does not work in all cases and is more than a little mysterious.)

C++11 allowed constants to be declared as static data members, deeming their declarations to not be definitions despite having initializers. Therefore, they did not need internal linkage, and C++17 was able to make such variables implicitly inline. (Similarly, C++14 introduced variable templates that, since they could be defined more than once, did not need the internal-linkage trick.) Modules do one better: by using a single definition of an entity for all translation units that need a definition of it, there is no need for `inline` to allow multiple definitions at all. (It still has its original meaning of encouraging the implementation to inline function calls, with the attendant ABI implications.)

Problem

However, declaring a const-qualified variable in a namespace gives it internal linkage even in a module unit. This prevents it from being used in client translation units:

```
// TU #1
module A:B;
const int dimensions=3;

// TU #2
module A;
import std;
import :B;
using vector=std::array<double,dimensions>;  // error: lookup failed
```

(Only clients in the same module are affected, since `export` assigns external linkage instead.) Moreover, such a variable cannot be used in an inline function or function template that does not itself have internal linkage:

```
// TU #1
export module A;
const double delta=0.01;
template<class F>
export double derivative(F &&f,double x) {
  return (f(x+delta)-f(x))/delta;
}

// TU #2
import A;
double d=derivative([](double x) {return x*x;},2);  // error: names delta
```

(The `get_zero` example above works in a module only because of another special rule ([basic.link]/14.4).)

This internal linkage can be avoided by adding `inline`, `extern`, or `export`, but each is confusing in this case:

- `inline` because its only standard effect is changing the linkage,
- `extern` because it's applied to a definition and grants *module* linkage rather than external linkage, and
- `export` because removing it in other circumstances does not affect clients within the same module.

Having to use any of these is an obstacle for converting a header-based library (that defines internal constants without `inline`) into a module.

Proposal

As a defect report against C++20, disable the special case for const-qualified variables in the module purview of importable module units, determining their linkage in the same fashion as for any other variable there. For compatibility, do not change the behavior in non-importable module units: they cannot provide definitions to any other translation unit, and converting existing *source* files to (non-partition) module implementation units should

not introduce conflicts between their internal-linkage constants. (It would of course be trivial to instead apply this change to all of the purview of any named module if desired.)

The obvious argument against such a change is that we do not want distinct language rules for module units and other translation units; EWG has rejected previous proposals suggesting such, despite the obvious benefits for modernization. However, in a practical sense such a distinction is the status quo: a const-qualified global variable in a header file can be used by its clients (albeit with the aforementioned ODR trap), but the same variable declared in an importable module unit cannot be used by its clients at all (nor even by its own inline functions). The fact that, with this change, such a variable changes from internal linkage to module linkage when changing from a header file to a module is part of the feature of avoiding ODR problems by defining things but once.

Wording

Relative to N4928.

#[basic.link]

Change bullet (3.2):

- a non-template variable of non-volatile const-qualified type, unless

    1. it is declared in the purview of a module interface unit (outside the *private-module-fragment*, if any) or module partition, or
    2. it is explicitly declared `extern`, or
    3. it is inline ~~or exported~~, or
    4. it was previously declared and the prior declaration did not have internal linkage; or

[*Drafting note*: The *private-module-fragment* exception follows /17. One could factor out a definition for an “importable declaration” or so. — *end drafting note*]